

FASE 1

Descrição da Arquitetura

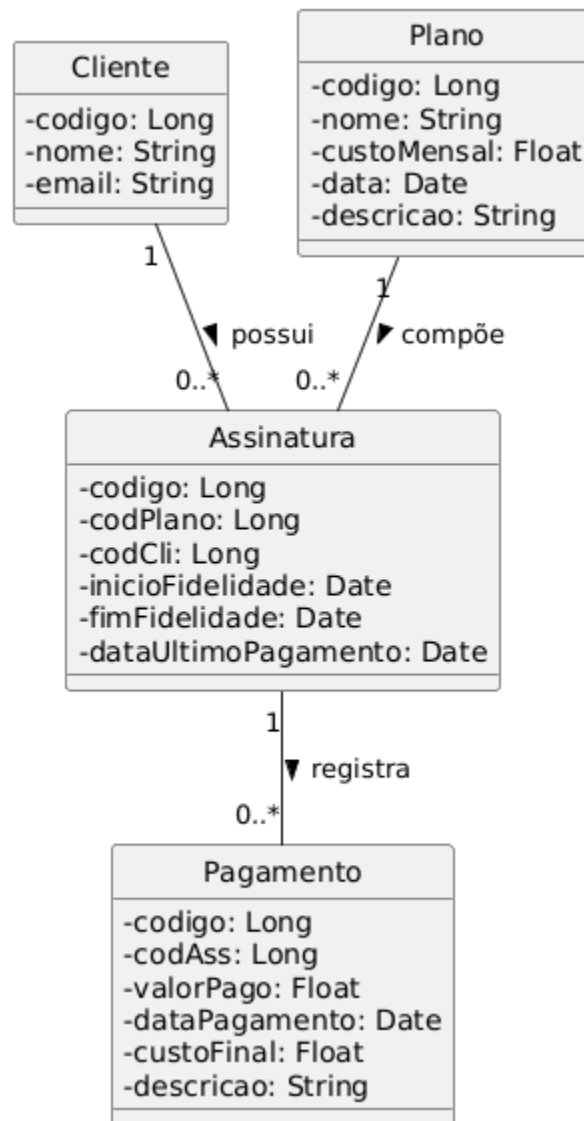


Diagrama UML: A modelagem ilustra a separação clara entre as camadas de Domínio, Aplicação, Adaptadores de Interface e Infraestrutura.

Atendimento aos Princípios SOLID: O Princípio da Responsabilidade Única (SRP) foi garantido separando as regras de negócio nos Casos de Uso e o acesso a dados nos repositórios. O Princípio da Inversão de Dependência (DIP) foi aplicado fazendo com que as

camadas superiores dependam de abstrações (interfaces), e não de implementações concretas do banco.

Arquitetura Limpa e Padrões de Projeto: O projeto segue rigorosamente a Regra de Dependência, onde as regras de negócio não conhecem detalhes de framework ou UI. O padrão *Repository* foi adotado para centralizar e abstrair as operações de persistência, enquanto a Injeção de Dependências facilita o acoplamento fraco.

Conclusão e Desafios: O principal desafio no desenvolvimento foi garantir que as regras de negócio ficassem totalmente agnósticas ao banco de dados. Isso foi resolvido com sucesso através da criação de portas e adaptadores, resultando em um sistema escalável, de fácil manutenção e altamente testável.

Orientações de Execução

Banco de Dados: O sistema utiliza o SQLite, escolhido por sua leveza, estrutura baseada em arquivos locais e por não exigir a configuração de um servidor externo, facilitando a validação e os testes da arquitetura.

Tecnologias Utilizadas: O projeto foi estruturado utilizando TypeScript, junto com o SQLite para a camada de persistência

1. Instalar as Dependências

```
npm install
```

2. Criar o Banco de Dados

```
npx prisma db push
```

3. Popular o Banco (Seed)

```
npm run seed
```

4. Iniciar o Servidor

npm run dev